Orbita_Ai Setup Guide

Build a private chat website that runs on your own laptop. Any phone or computer on your Wi-Fi can open it, ask questions, hear spoken replies, and ask for live weather. No subscription and no usage limit.

Contents

1. What you will build
2. Install the tools
3. Build the AI program
4. Download the AI model
5. Add the website file
6. Set up nginx
7. Start the AI
8. Keep the laptop awake
9. Open the website
10. Check that it works
11. Weather, effort levels, and voice
12. Install the Piper voice
13. Limits and settings
14. After a shutdown
15. Troubleshooting
16. Security notes
17. Remove everything

---

1. What you will build

Part	What it is
AI engine	llama.cpp, built on your laptop
AI model	Qwen2.5 1.5B Instruct (4-bit, about 1.1 GB)
Website	One file, `index.html`, named Orbita_Ai
Web server	nginx, which makes the site reachable at the laptop's address
Extras	Live weather (Open-Meteo), Piper voice, Effort switch (Low, Medium, High, Max)

What you need

- A laptop with Linux Mint (tested on 4 GB RAM, AMD A6-9220e, HDD)
- Internet for the setup downloads
- The `index.html` file for the Orbita_Ai site

Before you start

- Everything in the code boxes is typed into **Terminal**.
- Paste with **Ctrl + Shift + V**. Plain Ctrl + V does not paste in Terminal.
- Type your laptop password when asked. The letters stay invisible, which is normal.
- In this guide the username is `renz`. If yours is different, check it with `whoami` and swap it wherever you see `renz`.

This setup is yours to change. The steps work as written, but you can adjust these parts to fit your laptop and taste:

You can change	Where to look
Username (`renz`)	Parts 6, 7, and 12
AI model (smaller is faster, bigger is smarter)	Parts 4 and 13
CPU cores (`-t`), chat memory (`-c`), questions at once (`-np`)	Parts 7 and 13
Effort levels (reply length and history)	Part 13
Voice (speed, tone, or a different voice)	Part 12
Login on or off	Part 6
Site name and design	`index.html`
Names (Orbita_Ai, `ai-site` , `models` , `piper`)	Wherever they appear in the commands and files
Service and config file names (`laptop-ai` , `piper-voice` , `orbita.conf`)	Parts 6, 7, 12, 14, and 17

One rule: change a name only if you change it **everywhere it is used** (commands, service files, and the cleanup in Part 17). A name that does not match will break the setup. Keep `index.html` named exactly that, because the server looks for it.

---

2. Install the tools

```
sudo apt update
sudo apt install -y git build-essential cmake curl nginx apache2-utils
```

> Type `build-essential` with a dash and no space, and spell `nginx` as n-g-i-n-x. One typo makes the whole command fail. Pasting avoids this.

---

3. Build the AI program

```
cd ~
git clone https://github.com/ggerganov/llama.cpp
cd llama.cpp
cmake -B build
cmake --build build --config Release -j2
```

- The last command takes **20 to 60 minutes** on an old laptop.
- A percentage counts up. It is finished at 100% when the `$` prompt returns.
- Keep the laptop plugged in and the window open.
- Yellow warnings about ccache and OpenSSL are harmless.

The download is only source code. This step turns it into a program (`llama-server`). Everything is created inside the `llama.cpp` folder, so nothing else on your system changes.

---

4. Download the AI model

- [ ] In Firefox, go to **huggingface.co** and open **Qwen/Qwen2.5-1.5B-Instruct-GGUF**
- [ ] Click **Files and versions**
- [ ] Download `qwen2.5-1.5b-instruct-q4_k_m.gguf` (about 1.12 GB). No account needed. Ignore the other versions.

Then move it into a models folder:

```
mkdir -p ~/models
mv ~/Downloads/qwen2.5-1.5b*q4_k_m.gguf ~/models/
ls -lh ~/models
```

The last command should list one file of about 1.1 GB. If it says "No such file", run `ls -lh ~/Downloads` to see what the file is really called.

---

5. Add the website file

```
mkdir -p ~/ai-site
mv ~/Downloads/index.html ~/ai-site/
ls ~/ai-site
```

> The file must stay named `index.html`. The server looks for that exact name. Orbita_Ai is the name shown inside the page, not the file name.

---

6. Set up nginx

nginx receives visits on the normal web port (80) and passes them to the AI, which stays hidden on the laptop itself.

```
sudo rm -f /etc/nginx/sites-enabled/default
sudo nano /etc/nginx/conf.d/orbita.conf
```

Paste this, save with **Ctrl + O**, **Enter**, then close with **Ctrl + X**:

```
server {
    listen 80;

    location /tts {
        proxy_pass http://127.0.0.1:5000/;
        proxy_read_timeout 120s;
    }

    location / {
        proxy_pass http://127.0.0.1:8080;
        proxy_buffering off;
        proxy_read_timeout 300s;
    }
}
```

The `/tts` section sends voice requests to the Piper voice server from Part 12. It does no harm if you set up the voice later.

Check the file, restart nginx, and open the firewall:

```
sudo nginx -t
sudo systemctl restart nginx
sudo ufw allow 80
```

`nginx -t` should say syntax is ok. If `ufw` says the status is inactive, that is fine.

Optional: add a login

Without a login, anyone on your Wi-Fi can use the AI. To require a username and password:

```
sudo htpasswd -c /etc/nginx/.htpasswd renz
```

Add these two lines inside `location / {`:

```
auth_basic "Orbita_Ai";
auth_basic_user_file /etc/nginx/.htpasswd;
```

Then run:

```
sudo nginx -t
sudo systemctl reload nginx
```

- If the two passwords you type do not match, you get "password verification error". Run the command again.
- To remove the login later, delete those two lines and reload nginx.

---

7. Start the AI

Quick test (optional)

```
cd ~/llama.cpp
./build/bin/llama-server -m ~/models/qwen2.5-1.5b-instruct-q4_k_m.gguf --path ~/ai-site --host 127.0.0.1 --port 8080 -c 4096 -np
1 -t 2
```

- Wait for the line `listening on http://127.0.0.1:8080`, then open Firefox at `http://localhost`.
- Press **Ctrl + C** to stop it before the next step. If you close this window, the AI stops.
- Set `-t` to your CPU core count (see it with `nproc`).

Run it in the background (recommended)

This lets you close Terminal, and starts the AI by itself every time the laptop turns on.

```
sudo nano /etc/systemd/system/laptop-ai.service
```

Paste this, replacing `renz` if your username is different:

```
[Unit]
Description=Orbita_Ai local AI server
After=network.target

[Service]
User=renz
ExecStart=/home/renz/llama.cpp/build/bin/llama-server -m /home/renz/models/qwen2.5-1.5b-instruct-q4_k_m.gguf --path
/home/renz/ai-site --host 127.0.0.1 --port 8080 -c 4096 -np 1 -t 2
Restart=always

[Install]
WantedBy=multi-user.target
```

Save, then turn it on and check it:

```
sudo systemctl daemon-reload
sudo systemctl enable --now laptop-ai
sudo systemctl status laptop-ai
```

You want to see green `active (running)`. Press **Q** to leave the status screen. The model takes one to two minutes to load.

What the flags do

Flag	Meaning
`-c 4096`	Gives each chat enough room for long Max-effort answers
`-np 1`	Handles one question at a time (faster, uses less memory)
`-t`	Number of CPU cores to use. Set it to the number `nproc` prints

---

8. Keep the laptop awake

A laptop that is truly asleep cannot run the AI. The screen can still turn off. Block sleep and lid-close sleep:

```
sudo systemctl mask sleep.target suspend.target hibernate.target hybrid-sleep.target
sudo nano /etc/systemd/logind.conf
```

In that file, set these two lines (remove the `#` at the start, or add them at the bottom if missing):

```
HandleLidSwitch=ignore
HandleLidSwitchExternalPower=ignore
```

Then run:

```
sudo systemctl restart systemd-logind

gsettings set org.cinnamon.settings-daemon.plugins.power sleep-inactive-ac-type 'nothing'
gsettings set org.cinnamon.settings-daemon.plugins.power lid-close-ac-action 'nothing'
gsettings set org.cinnamon.settings-daemon.plugins.power lid-close-battery-action 'nothing'
```

> Keep the laptop plugged in and on a hard surface with airflow. An old laptop running all day with the lid closed can get warm.

To restore sleep later:

```
sudo systemctl unmask sleep.target suspend.target hibernate.target hybrid-sleep.target
```

---

9. Open the website

On the laptop: open Firefox and go to `http://localhost`.

On a phone or another computer:

- [ ] On the laptop, run `hostname -I`. It prints numbers like `192.168.1.50`. Use the first set.
- [ ] On the other device, open `http://192.168.1.50` (use your own numbers).

Make sure the device is on the **same Wi-Fi**, any VPN is **off**, and you do **not** add `:8080`.

Why `localhost` fails on the phone: `localhost` always means "this device itself". On your phone it points to the phone, which has no AI. Always use the laptop's numbers on other devices.

These numbers can change after a router restart. If the page stops loading, run `hostname -I` again, or reserve a fixed IP for the laptop in your router settings.

---

10. Check that it works

Check	What you should see
Text under the page title	**Online**. "Loading model..." means wait a bit. "Server not reachable" means it is off.
`sudo systemctl status laptop-ai`	Green `active (running)`
`sudo systemctl status piper-voice`	Green `active (running)` (once Part 12 is done)
`curl http://127.0.0.1:8080/health`	`{"status":"ok"}`
`sudo journalctl -u laptop-ai -f`	New lines appear while you ask a question (Ctrl + C to stop)
`top`	`llama-server` near the top with high CPU while it answers (Q to exit)

---

11. Weather, effort levels, and voice

Live weather

Ask things like "What's the weather in Manila?" or "Will it rain in London tomorrow?". The page finds the city, fetches live data (now plus 3 days) from Open-Meteo, and the AI explains it. A small grey line above the answer shows which place was used.

- Always name the city after "in", "at", or "for".
- The device you are using needs internet for this.

Effort levels

The **Effort** button in the top right cycles Low, Medium, High, Max. The page remembers your choice and starts on Max.

Level	Longest reply	Style
Low	about 90 words	One or two short sentences. Fastest.
Medium	about 220 words	Clear and concise.
High	about 500 words	Thorough, step by step.
Max	about 1,000 words	Most thorough: step by step, edge cases, examples, double-checks its reasoning, says when it is unsure. Uses the steadiest setting and remembers the last 10 messages.

Longer answers take longer on an old laptop. Use Low or Medium for quick questions and keep Max for hard ones. Turning **Voice off** also makes answers faster, because the voice shares the same CPU.

Voice

- **Voice on / Voice off** (top right): the AI speaks its answers with the Piper voice, sentence by sentence while it is still writing. When Voice is on, the AI is also told to write the way people talk (short sentences, no lists or emojis). Needs the voice from Part 12.
- The page cleans the text before speaking: "25°C" is said as "25 degrees Celsius", "40%" as "40 percent", and links and emojis are skipped.
- **Mic button** (next to Send): tap, speak, and the question sends itself. This uses the browser's own speech recognition.
- **Firefox cannot do voice input.** Use Chrome or Safari for the mic.
- Browsers allow the mic only on secure pages. `http://localhost` counts, but `http://192.168...` does not, so on a phone you will see a message saying so.
- Chrome sends your voice to Google to turn it into text, so the device needs internet for the mic.

Making the mic work on an Android phone (Chrome)

- [ ] Open `chrome://flags/#unsafely-treat-insecure-origin-as-secure`
- [ ] Type your laptop's address, for example `http://192.168.1.50`
- [ ] Set it to **Enabled** and tap **Relaunch**
- [ ] Open the site, tap the mic, and allow the microphone

An iPhone has no such flag. The proper fix there is HTTPS (for example through Tailscale).

---

12. Install the Piper voice

Piper is a small voice model that runs on your laptop and sounds much more natural than the browser's built-in voice. This guide uses the British male voice **Alan**.

Step 1: Check your CPU is supported

```
lscpu | grep -o -w avx
```

If it prints `avx`, continue. If it prints nothing, Piper may crash on this CPU.

Step 2: Install Piper

```
sudo apt install -y python3-venv python3-pip
python3 -m venv ~/piper-env
~/piper-env/bin/pip install "piper-tts[http]"
```

Step 3: Download the voice

```
mkdir -p ~/piper
cd ~/piper
~/piper-env/bin/python3 -m piper.download_voices en_GB-alan-medium
ls -lh ~/piper
```

You should see `en_GB-alan-medium.onnx` (about 60 MB) and `en_GB-alan-medium.onnx.json`.

Step 4: Test the voice

```
~/piper-env/bin/python3 -m piper -m en_GB-alan-medium --data-dir ~/piper -f ~/test.wav -- 'Good evening. All systems are online.'
```

Double-click `test.wav` in your Home folder to hear it.

Step 5: Create the voice server file

The web server that comes with Piper did not work here: it returned a web page instead of sound (405 Method Not Allowed). This small replacement does one thing: it turns text into a WAV sound file.

Paste the whole block at once, from the first line to the line that says `EOF`. If your username is not `renz`, change it in the `MODEL` line. It prints nothing when it works.

```
cat > ~/piper/voice_server.py << 'EOF'
import io
import wave
from flask import Flask, request, Response
from piper import PiperVoice

try:
    from piper import SynthesisConfig
except Exception:
    SynthesisConfig = None

MODEL = "/home/renz/piper/en_GB-alan-medium.onnx"

# Tune these to change how the voice sounds
LENGTH_SCALE = 0.92   # below 1 = quicker and more energetic, above 1 = slower
NOISE_SCALE = 0.85    # higher = more varied, expressive tone
NOISE_W_SCALE = 1.0   # higher = more varied rhythm
VOLUME = 1.0

voice = PiperVoice.load(MODEL)
syn_config = None
if SynthesisConfig is not None:
    try:
        syn_config = SynthesisConfig(
            volume=VOLUME,
            length_scale=LENGTH_SCALE,
            noise_scale=NOISE_SCALE,
            noise_w_scale=NOISE_W_SCALE,
        )
    except TypeError:
        syn_config = None

app = Flask(__name__)


def make_wav(text):
    buf = io.BytesIO()
    with wave.open(buf, "wb") as wf:
        if hasattr(voice, "synthesize_wav"):
            if syn_config is not None:
                voice.synthesize_wav(text, wf, syn_config=syn_config)
            else:
                voice.synthesize_wav(text, wf)
        else:
            voice.synthesize(text, wf)
    return buf.getvalue()


@app.route("/", methods=["GET", "POST"])
def speak():
    if request.method == "POST":
        data = request.get_json(silent=True) or {}
        text = data.get("text", "")
    else:
        text = request.args.get("text", "")
    text = (text or "").strip()
    if not text:
        return Response("no text", status=400)
    return Response(make_wav(text), mimetype="audio/wav")


if __name__ == "__main__":
    app.run(host="127.0.0.1", port=5000, threaded=False)
EOF
```

Step 6: Run it in the background

```
sudo nano /etc/systemd/system/piper-voice.service
```

Paste this (change `renz` if your username is different), then save with **Ctrl + O**, **Enter**, **Ctrl + X**:

```
[Unit]
Description=Piper voice server
After=network.target

[Service]
User=renz
ExecStart=/home/renz/piper-env/bin/python3 /home/renz/piper/voice_server.py
Restart=always

[Install]
WantedBy=multi-user.target
```

```
sudo systemctl daemon-reload
sudo systemctl enable --now piper-voice
sleep 8
sudo systemctl status piper-voice --no-pager
```

You want green `active (running)`.

Step 7: Test it through the website path

```
curl -s -X POST -H 'Content-Type: application/json' -d '{"text":"Good evening. All systems are online."}' http://127.0.0.1/tts -o ~/t.wav
file ~/t.wav
paplay ~/t.wav
```

`file` should say `RIFF ... WAVE audio`, and you should hear the voice. Then open the site, refresh with **Ctrl + Shift + R**, and press the voice button until it says **Voice on**.

Tuning the voice

Edit the numbers near the top of the file, then restart the service:

```
nano ~/piper/voice_server.py
sudo systemctl restart piper-voice
```

Sound	Settings
More lively	`LENGTH_SCALE = 0.88`, `NOISE_SCALE = 0.9`
Calmer, more butler-like	`LENGTH_SCALE = 1.0`, `NOISE_SCALE = 0.7`

If it sounds jittery or odd, bring `NOISE_SCALE` back toward 0.7.

To try another British voice, download it the same way as Step 3 (for example `en_GB-northern_english_male-medium`), change the `MODEL` line, and restart the service. Samples of every voice are on the Piper samples page.

---

13. Limits and settings

There is no quota or cost. The limits come from the laptop's memory and speed, and you set them.

Setting	Where	Current value
Memory per chat (question + history + reply)	`-c` in the AI service file	4096 tokens (about 3,000 words)
Longest single reply	`LEVELS` table in `index.html`	120 / 300 / 700 / 1400 tokens (Low / Medium / High / Max)
Past messages sent each time	`LEVELS` table in `index.html`	4 / 6 / 8 / 10 messages
Questions handled at once	`-np` in the AI service file	1 (one at a time)
Voice speed and expressiveness	Top of `~/piper/voice_server.py`	Editable (Part 12)

A token is roughly three-quarters of a word.

To change the effort levels: edit the `LEVELS` table near the top of the script in `index.html`, replace the file in `~/ai-site`, and refresh with **Ctrl + Shift + R**. No restart is needed.

To change `-c`, `-np`, or `-t`: edit the service file, then restart:

```
sudo nano /etc/systemd/system/laptop-ai.service
sudo systemctl daemon-reload
sudo systemctl restart laptop-ai
```

To make answers faster:

- Use Low or Medium effort
- Turn Voice off
- Set `-t` to your core count
- Close other apps
- For a big jump in speed, download **Qwen2.5-0.5B-Instruct-GGUF** (`q4_k_m`, about 400 MB), put it in `~/models`, and point the service file at it. It is roughly 2 to 3 times faster but less smart.

---

14. After a shutdown

The AI, the voice, and nginx all start by themselves when the laptop powers on.

- [ ] Log in
- [ ] Wait one to two minutes for the model to load
- [ ] Open the site. When the status under the title says **Online**, it is ready.

A full shutdown stops the AI. A sleeping laptop also stops it, which is why Part 8 matters.

To confirm all three start on boot:

```
systemctl is-enabled laptop-ai piper-voice nginx
```

It should print `enabled` three times.

---

15. Troubleshooting

Problem	Fix
Red `E: Unable to locate package` during install	A typo (space in `build essential`, or `ngix`). Paste the command from Part 2.
`git: command not found`	The install in Part 2 failed. Run it again and look for red `E:` lines.
`cd~: command not found`	A space is needed: `cd ~`
`mv: cannot stat ... No such file`	The file name is wrong. Use the `*` command in Part 4, or run `ls ~/Downloads`.
502 Bad Gateway	The AI is not running. Run `sudo systemctl restart laptop-ai`, then wait a minute.
Page works on the laptop but not the phone	Use `http://` plus the numbers from `hostname -I`, not `localhost`. Same Wi-Fi, VPN off, no `:8080`.
Phone worked before but stopped	The laptop's numbers changed. Run `hostname -I` again.
Answers say "Server answered 429"	A rate limit was added in nginx. Remove any `limit_req` lines, or wait a few seconds.
Voice input says it needs a secure page	Expected on `http://192.168...`. Use the Android flag in Part 11, or HTTPS.
Service shows `failed`	Run `sudo journalctl -u laptop-ai -n 30` to see why. Check the model file name and the paths in the service file.
Voice returns a web page (405) or `file ~/t.wav` says HTML	The stock Piper web server does not work with this page. Use the `voice_server.py` file from Part 12, Step 5.
Page says "The voice is not working: the voice server answered ..."	Check `sudo systemctl status piper-voice` and `sudo journalctl -u piper-voice -n 20 --no-pager`. Confirm the `/tts` section is in the nginx file (Part 6).
Page says "The voice audio could not be played"	The voice server sent something that is not sound. Run the Step 7 test in Part 12 and check that `file` says WAVE audio.
Answers are very slow	Normal on an old CPU. Close other apps, set `-t` to your core count, or use the 0.5B model.

---

16. Security notes

- This is meant for your **home Wi-Fi**. Without a login, anyone connected to it can use your AI.
- Do **not** open any ports on your router.
- To reach it from outside your home, install **Tailscale** on the laptop and your devices. It encrypts the connection and keeps strangers out.

```
curl -fsSL https://tailscale.com/install.sh | sh
sudo tailscale up
```

- Do not put the AI on the public internet without a login and encryption.

---

17. Remove everything

Nothing else on the laptop depends on this, so you can remove it any time. Copy these exactly, because `rm -rf` deletes permanently with no Trash.

```
sudo systemctl disable --now laptop-ai piper-voice
sudo rm /etc/systemd/system/laptop-ai.service /etc/systemd/system/piper-voice.service
sudo systemctl daemon-reload
rm -rf ~/llama.cpp ~/models ~/ai-site ~/piper ~/piper-env
sudo rm /etc/nginx/conf.d/orbita.conf
sudo systemctl restart nginx
sudo ufw delete allow 80
```

Optional: remove the tools with `sudo apt remove nginx apache2-utils`. The build tools (git, cmake) are harmless and useful, so you can keep them. If you installed Tailscale: `sudo apt remove tailscale`.